

A. ADDITIONAL RESULTS

A.1 Per-Task Success and Statistical Significance

For every task we use the same SCORE hyperparameters, train a single seed, and select a single checkpoint for real-world evaluation using simulation results alone.

To test whether the gains hold across tasks, we run a Cochran–Mantel–Haenszel (CMH) test stratified by task, comparing each SCORE variant against Residual-RL, the strongest baseline. SCORE significantly outperforms Residual-RL (CMH statistic = 28.77, $p < 0.001$, estimated common odds ratio = 7.53). SCORE ($\epsilon=0$), which uses latent steering alone, also significantly outperforms Residual-RL (CMH statistic = 7.80, $p = 0.005$, estimated common odds ratio = 2.383).

A.2 Simulation Success and Sim-to-Real Transfer

Fig. 14 reports simulation success for each task, and Fig. 9 compares each method’s simulated and real-world success, averaged over the eight tasks; the dashed line marks equal success in the two domains. Simulated gains transfer to hardware only when policy improvement is constrained to the support of the base policy. Simulation overestimates the real-world success of FPO and RialTo by 45.6 and 37.6 points, and of Residual-RL by 8.5 points: all three improve in simulation in ways that do not survive deployment. SCORE ($\epsilon=0$) and SCORE instead succeed 6.3 and 13.2 points *more* often in the real world than in simulation, because real-world resets are narrower than the heavily randomized simulation, so their simulated success is a conservative estimate of what transfers. The base policy is never trained in simulation and succeeds only 1.8% of the time under the broader simulated resets, so we omit it from Fig. 9. Across the five methods trained in simulation, ranking by average simulated success gives the same order as ranking by average real-world success, although SCORE and SCORE ($\epsilon=0$) (76.2 vs. 76.1) and RialTo and FPO (62.4 vs. 61.8) are nearly tied in simulation.

A.3 Does SCORE stay within the support?

The main text reports the nearest-neighbor containment distances; we note two details here. First, flow steering enforces a support constraint only if the steering latents stay in high-density regions of the prior, and the flow model’s push-forward faithfully models the data [33]. Across 1024 simulated Cube Pinch evaluations, 98.6% of steering latents fall inside the central 99% of the prior’s norm distribution, and we additionally clip latents to $[-3, 3]$. Second, the distances are best read against the base policy’s own variability: a second independent sample from the base policy sits at 0.037, so SCORE ($\epsilon=0$) (0.033) is within the base policy’s own sampling noise, while SCORE (0.046) and Residual-RL (0.048) sit just outside it.

A.4 Robustness to Dynamics Mismatch

To test SCORE under deliberate sim-to-real mismatch, we train Cube Pinch policies in simulations with one physical parameter changed at a time: 1–3 steps of added action latency, modified controller gains, a cube twice the real size, and reduced cube friction. Each policy is deployed directly on the real robot and succeeds on 15/15 trials (100%).

A.5 Retry Data Without the Residual

Sec. V-B shows that retry data lets SCORE reach 100% on Cube Pinch, compared with 40% when the base policy is trained on optimal demonstrations only. On the base policy without retry data (30% real-world success), SCORE ($\epsilon=0$) improves only to 35%, while SCORE and Residual-RL both reach 40%. The gains come from more precise grasps; none of these policies exhibits recovery behavior. The bounded residual is too small to create recovery behavior that is absent from the base policy’s support.

A.6 Engineering Effort

Beyond collecting demonstrations, the only per-task human effort is scanning the task objects and setting up the simulated scene (Appendix D.4), which took 10–30 minutes per task. All tasks share the same SCORE hyperparameters and are trained from sparse success rewards (Appendix B.4).

B. EXPERIMENT DETAILS

B.1 Hardware and Control Setup

Fig. 15 shows our real-world experimental setup. We use a Franka Research 3 (FR3) robot arm equipped with a LEAP hand, constructed following the hardware setup provided by [45]. The workspace is observed by a single external Orbbec Femto Bolt depth camera mounted facing the front of the robot.

We deploy all policies using the DROID [46] stack, which builds on Polymetis [47], and run closed-loop inference at 10 Hz. At each control step, the policy receives the current observation and predicts absolute joint position targets, which are executed by the robot’s joint-impedance controller.

For 3D perception, we first crop the raw depth point cloud to a predefined workspace bounding box containing the table and relevant task objects. We then remove the tabletop and background points using RANSAC-based plane fitting and express the remaining points in the robot base frame. This filtered point cloud is used as the input to the policy.

B.2 Dexterous Manipulation Tasks

Shown in Fig. 3, we evaluate on a suite of eight dexterous manipulation tasks designed to stress different aspects of multi-fingered control, including precise grasping, repeated in-hand motion, non-prehensile interaction, and secondary object interactions. Table II summarizes each task. We provide additional videos and qualitative examples on the project website: <https://score-anon.github.io>.

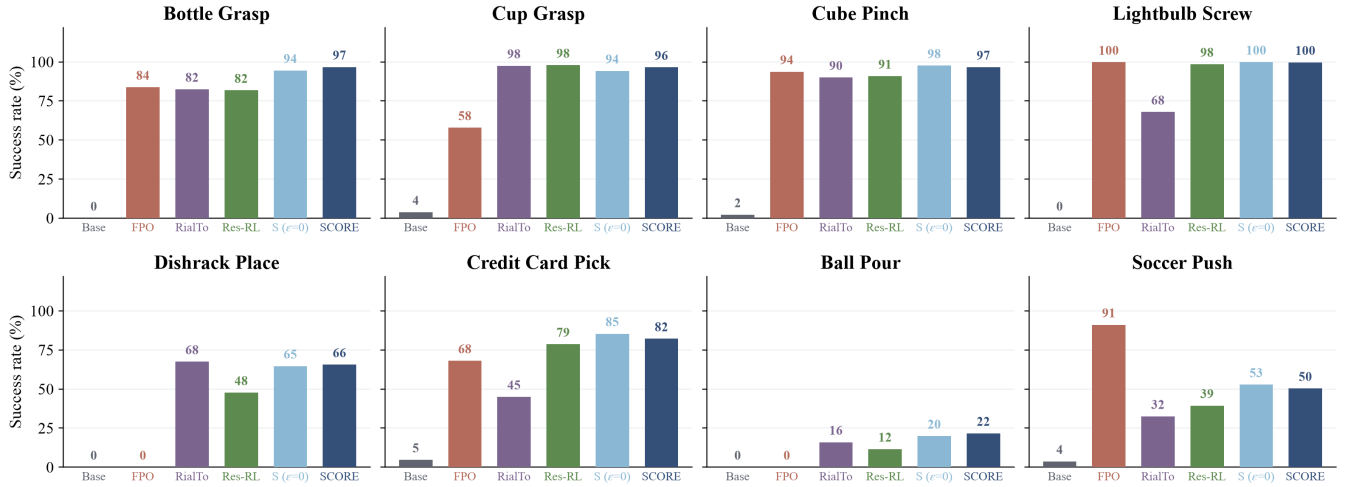


Fig. 14. **Per-task simulation success rates.** Each method is evaluated over 4096 simulated environments per task, with resets sampled uniformly from the simulation ranges in Table IV; RialTo is evaluated after distillation. Because simulation is heavily domain-randomized, simulated success is often lower than real-world success for policies that transfer, especially SCORE. S ($\epsilon=0$) abbreviates SCORE ($\epsilon=0$).

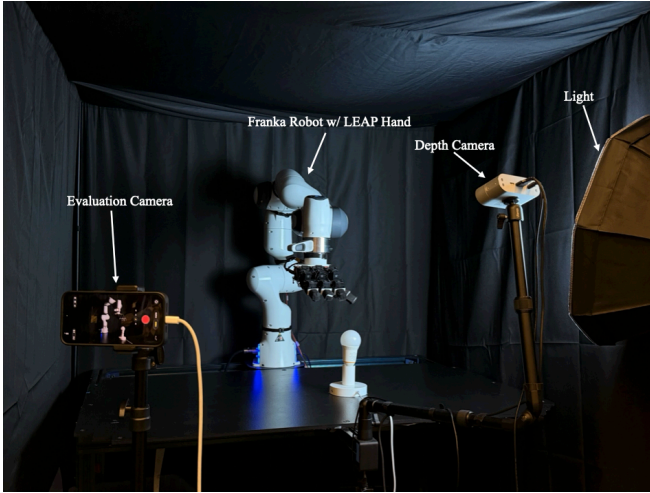


Fig. 15. **Real-world Robot Setup**

B.3 Data Collection

We collect real-world demonstrations using an Apple Vision Pro teleoperation interface. The system tracks the operator’s hand motion and end-effector motion using keypoints, and retargets these motions to the Franka arm and LEAP hand. This allows an operator to provide coordinated arm and finger commands for dexterous, contact-rich manipulation tasks.

The amount of data varies by task. Table II reports the number of real-world demonstrations collected for each task. Furthermore, for simple grasping tasks, suboptimal behavior followed by retry data was added to encourage retry behavior depicted in Fig. 16.

B.4 Rewards

We use minimal reward engineering across all tasks. The task-specific reward for each task is a sparse signal equal to the same success condition used for evaluation, with no

TABLE II
REAL-WORLD DEMONSTRATIONS PER TASK

Task	Demos	Task type
Bottle Grasp	100	Multi-finger grasp and lift
Cup Grasp	100	Side grasp and lift
Cube Pinch	100	Precise fingertip pinch
Lightbulb Screw	90	Repeated contact and twisting
Credit Card Pick	130	Thin-object grasp against a shelf
Dishrack Place	100	Precise object placement
Ball Pour	100	Dynamic manipulation of a ball
Soccer Push	170	Non-prehensile pushing with retries
Total	890	

dense shaping or per-task tuning. The only additional terms are lightweight joint-velocity and action-rate penalties that are shared across every task. Table III lists the reward and success criterion for each task.

B.5 Evaluation Protocol

For each task, we evaluate policies under randomized initial conditions rather than a single fixed reset. At the beginning of each episode, task-relevant object poses are sampled uniformly from task-specific reset ranges. Table IV compares the reset distribution used in simulation with the reset distribution used during real-world evaluation.

The simulation reset distribution is generally broader than the real-world evaluation reset distribution. In simulation, we can randomize object poses, scene elements, and physical parameters at scale. In the real world, resets are performed manually and are constrained by the physical workspace and by what can be reliably initialized by the operator. Unless otherwise specified, pose offsets are sampled uniformly from the intervals in Table IV.

We report task-specific success rates in Fig. 4.

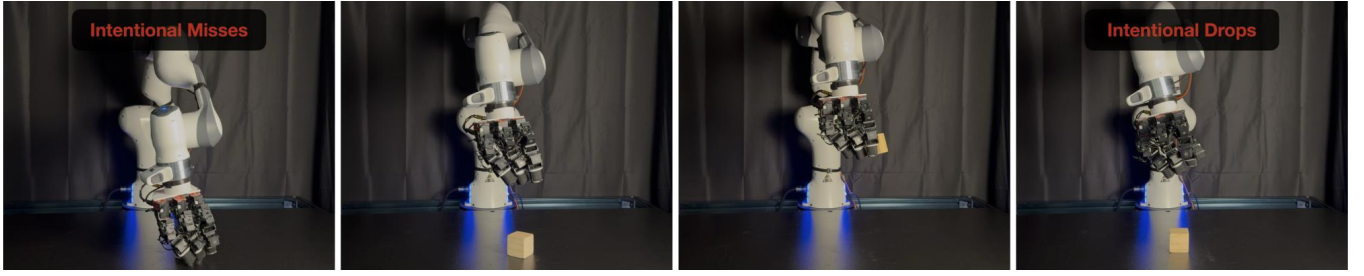


Fig. 16. **Retry Data Collection.** For the Cube Pinch and Bottle Grasp tasks, drops and misses followed by retries are captured in the dataset to encourage learning retry behavior.

TABLE III
TASK-SPECIFIC REWARDS AND SUCCESS CRITERIA

Task	Task-specific reward	Success criterion
Bottle Grasp	$10.0 r_{\text{success}}$: lift bottle to target height.	Object height $z \geq 0.25\text{m}$.
Cup Grasp	$10.0 r_{\text{success}}$: lift cup to target height.	Object height $z \geq 0.20\text{m}$.
Cube Pinch	$10.0 r_{\text{success}}$: lift cube to target height.	Object height $z \geq 0.20\text{m}$.
Lightbulb Screw	$10.0 r_{\text{success}}$: rotate lightbulb to the stable on-state.	Cumulative lightbulb rotation exceeds 2π .
Credit Card Pick	$10.0 r_{\text{success}}$: lift card to target height.	Card height $z \geq z_{\text{spawn}} + 0.18\text{m}$.
Dishrack Place	$50.0 r_{\text{success}}$: place plate in rack slot with desired pose.	Plate is inside the rack slot with desired position and orientation.
Ball Pour	$10.0 r_{\text{ball_in_cup}}$: place ping-pong ball inside big cup.	Ball is inside big cup: XY distance within 0.05m of cup center and Z position within cup height 0.12m .
Soccer Push	$10.0 r_{\text{ball_in_goal}}$: push ball inside goal bounding box.	Ball is inside goal box: relative to goal frame, $x \in [-0.07, 0.07]$, $y \in [-0.03, 0.03]$, $z \in [0.01, 0.08]$.

All tasks additionally use shared lightweight joint-velocity and action-rate penalties.

C. BASELINE IMPLEMENTATIONS

C.1 FPO

We adapt the FPO++ implementation from [14] to fine-tune our pre-trained flow matching behavior cloning policy [9]. The PointNet observation encoder is frozen, and RL gradients are applied only to the UNet denoising network. We use the same asymmetric actor-critic setup as SCORE: the actor receives deployment-compatible point cloud observations, while a separate MLP critic with hidden layers [256, 128, 64] and ELU activations receives privileged simulation state.

Following FPO++, we replace the standard PPO likelihood ratio with a CFM-loss-based policy ratio,

$$\rho = \exp(\mathcal{L}_{\text{CFM}}^{\text{old}} - \mathcal{L}_{\text{CFM}}^{\text{new}}),$$

where $\mathcal{L}_{\text{CFM}}^{\text{old}}$ is computed using the frozen policy from the previous update and $\mathcal{L}_{\text{CFM}}^{\text{new}}$ is computed using the current policy. We then optimize the PPO clipped surrogate objective with the asymmetric trust region variant used in the reference implementation. Unless otherwise stated, we use separate Adam optimizers for the actor and critic with learning rates 3×10^{-5} and 1×10^{-3} , respectively, clip range $\epsilon = 0.01$, GAE with $\gamma = 0.99$ and $\lambda = 0.95$, and 4096 parallel environments with 200 rollout steps per update.

We also attempted BC regularization, which anchors the fine-tuned UNet to the frozen behavior cloning policy by penalizing deviations in CFM loss on real demonstration

data. We found that this regularization introduced a sensitive tradeoff: large coefficients prevented meaningful policy improvement, while small coefficients behaved similarly to unregularized FPO. Since no tested coefficient improved the final tradeoff between simulation improvement and real-world transfer, we report unregularized FPO as the baseline.

We swept 150 FPO configurations across the axes in Table VI. CFM sample counts above 8 required reducing rollout length or minibatch size to stay within memory budget. More than 5 PPO epochs per update did not improve performance despite FPO++ using 16–32 for locomotion. No amount of BC regularization coefficient improved over unregularized FPO, so we omit it from main comparisons. Our final hyperparameters are reported in Table V.

C.2 RialTo

We implement a RialTo-style real-to-sim-to-real baseline [3]. For each task, we scan the deployment environment into simulation and apply inverse distillation: a perceptual policy trained on real demonstrations is rolled out in the scanned simulator, and successful rollouts are recorded with privileged state (joint state, object pose, task-specific features) to produce a privileged demonstration buffer.

We train the privileged teacher in two stages. First, we behavior-clone a state-based MLP policy on the inverse-distilled demonstrations using MSE loss. Second, we initialize a PPO policy from the BC checkpoint and fine-tune it in

TABLE IV
SIMULATION AND REAL-WORLD RESET DISTRIBUTIONS

Task	Simulation reset randomization	Real evaluation reset randomization
Bottle Grasp	Bottle pose is centered at (0.50, 0.00, 0.11) with $\Delta x = \pm 0.20\text{m}$, $\Delta y = \pm 0.30\text{m}$, and yaw sampled from $\{0, \pi\}$.	Evaluated over 54–59 trials per method. Randomization is same as simulation.
Cup Grasp	Cup pose is centered at (0.525, 0.10, 0.07) with $\Delta x \in [-0.075, 0.175]\text{m}$ and $\Delta y = \pm 0.10\text{m}$.	Evaluated over 45–60 trials per method. Cup is randomized only along the x axis. The real reset range is narrower than simulation, with approximately 0.075m removed from each side of the simulated x range, and no y randomization.
Cube Pinch	Cube pose is centered at (0.50, 0.00, 0.07) with $\Delta x = \pm 0.20\text{m}$ and $\Delta y = \pm 0.20\text{m}$.	Evaluated over 30–60 trials per method. Randomization is same as simulation.
Lightbulb Screw	Lightbulb pose is centered at (0.545, 0.13, 0.00) with $\Delta x = \pm 0.02\text{m}$, $\Delta y = \pm 0.03\text{m}$, and $\Delta z \in [-0.01, 0.02]\text{m}$. The bulb is initialized roughly 1.5 turns away from completion.	Evaluated over 30–39 trials per method. Lightbulb pose is fixed in the x and y directions during real evaluation; we unscrew the lightbulb one full rotation away from the stable yellow light.
Dishrack Place	Plate pose is centered at (0.58, -0.12, 0.05) with $\Delta x = \pm 0.055\text{m}$ and $\Delta y \in [-0.048, 0.05]\text{m}$. Rack pose is centered at (0.57, 0.14, 0.00) with $\Delta x = \pm 0.02\text{m}$ and $\Delta y = \pm 0.02\text{m}$.	Evaluated over 38–50 trials per method. Plate pose is randomized over the same range as simulation. The rack pose is fixed during real evaluation.
Credit Card Pick	Shelf pose is centered at (0.55, -0.20, 0.00) with $\Delta x \in [-0.05, 0.10]\text{m}$, $\Delta y = \pm 0.03\text{m}$, and $\Delta z \in [-0.01, 0.03]\text{m}$. The card receives an additional x offset in $[-0.05, 0.00]\text{m}$ on the shelf.	Evaluated over 30–40 trials per method. The card is fixed during real evaluation; we do not randomize the card pose.
Ball Pour	Small cup pose is centered at (0.54, -0.115, 0.07) with $\Delta x = \pm 0.08\text{m}$ and $\Delta y = \pm 0.04\text{m}$. Big cup pose is centered at (0.55, 0.12, 0.07) with $\Delta x = \pm 0.08\text{m}$ and $\Delta y = \pm 0.08\text{m}$.	Evaluated over 30–44 trials per method. The source cup is randomized only along the x axis. The target cup is fixed during real evaluation.
Soccer Push	Ball pose is centered at (0.53, -0.125, 0.035) with $\Delta x = \pm 0.15\text{m}$ and $\Delta y \in [-0.15, 0.10]\text{m}$.	Evaluated over 30–45 trials per method. Randomization is same as simulation.

All poses are in the robot base frame. Simulation generally uses a broader reset distribution, while real-world evaluation uses the same task setup as data collection with restricted object and scene randomization.

simulation with a BC-regularized objective:

$$\mathcal{L}_{\text{BC}} = \|\mu_{\pi}(s_{\text{demo}}) - a_{\text{demo}}\|_2^2,$$

where μ_{π} is the Gaussian policy mean. Unlike the original RialTo, we use a continuous joint-action space. We find 30 critic-only warmup rollouts necessary to prevent collapse of the base policy before enabling policy-gradient updates. We additionally run a sweep over the BC loss coefficient on Cube Pinch, finding 10k to be the best value, as shown in Fig. 6. All hyperparameters are reported in Table V.

C.3 Residual RL

We compare against a Residual RL baseline implemented by removing the noise prediction branch from SCORE while keeping all other components unchanged. The actor receives the same observations, the critic uses the same privileged state, and training uses the same RL objective, optimizer settings, and rollout configuration.

D. REAL-TO-SIM-TO-REAL PIPELINE

To ensure reproducibility, we provide full details of our real-to-sim-to-real pipeline, including model architecture, simulation design, and considerations for real world deployment. Our base policy is a conditional flow matching model trained on real-world demonstrations collected by a human. The policy uses a simple PointNet encoder together with a U-Net to parameterize the velocity field. It is conditioned on

the robot joint positions and a point cloud observation of the scene, and predicts absolute joint position targets. We then run RFS in simulation using sparse rewards to improve the base policy, and finally evaluate the improved policy in the real world.

D.1 Observation Space

To enable efficient policy improvement, our goal is to run massively parallel RL using the same observation space as the base policy. Direct vision-based observations, such as RGB or depth images, require computationally expensive rendering in simulation. In practice, we find that on a single NVIDIA L40S GPU, we can run at most 512 parallel environments with RGB or depth-image rendering when training simple MLP policies, and environment steps are significantly slower when performing rendering.

Instead, we sample points directly from the environment meshes and transform them using the ground-truth frame transforms of the robot and objects throughout the episode, as described by [48]. This gives the policy access to privileged point cloud observations in simulation while preserving the same point cloud observation structure used by the real-world policy. This representation allows us to run up to 4096 parallel environments per GPU with faster environment steps, substantially reducing wall-clock training time.

The full observation consists of the absolute joint positions of the robot’s 23 joints, with 7 joints for the Franka arm

TABLE V
HYPERPARAMETERS FOR THE RIALTO AND FPO BASELINES

RialTo		FPO	
Parameter	Value	Parameter	Value
<i>Stage 1: Inverse Distillation</i>		<i>Architecture</i>	
Policy class	MLP	Actor observation	PCD (deployment)
Hidden layers	[512, 512, 512]	Critic observation	Privileged state
Activation	Tanh	PointNet encoder	Frozen
Action space	Continuous joints	Denoising network	UNet (fine-tuned)
Loss	MSE	Critic hidden layers	[256, 128, 64]
Optimizer	Adam	Critic activation	ELU
Learning rate	3×10^{-4}	<i>Training</i>	
LR schedule	Cosine annealing	Policy ratio	CFM-loss-based ρ
Training epochs	500	Trust region	Asymmetric PPO clip
<i>Stage 2: PPO Fine-tuning</i>		Actor optimizer	Adam
Policy class	Gaussian MLP	Critic optimizer	Adam
Hidden layers	[512, 512, 512]	Actor learning rate	3×10^{-5}
Learning rate	1×10^{-4}	Critic learning rate	1×10^{-3}
Clip range	0.2	Clip range ϵ	0.01
Discount γ	0.99	Discount γ	0.99
GAE λ	0.95	GAE λ	0.95
Value loss coefficient	0.1	Parallel environments	4096
BC loss coefficient	10000	Rollout steps	200
Initial log σ	-3.0	BC regularization	None
Parallel environments	4096		
Rollout steps	200		
Batch size	4096		
PPO epochs per update	1		
Critic warmup rollouts	30		

TABLE VI
FPO HYPERPARAMETER SWEEP

Parameter	Values swept
Actor learning rate	$\{1 \times 10^{-5}, 3 \times 10^{-5}\}$
Critic learning rate	$\{1 \times 10^{-4}, 1 \times 10^{-3}\}$
CFM samples per action	$\{1, 4, 8, 16, 32\}$
PPO epochs per update	$\{1, 5, 10, 16, 32\}$
Trust region mode	asymmetric PPO, standard clip, CFM-loss clamp, neg.-advantage clamp, ST-estimator log-ratio clamp
Critic warmup updates	$\{0, 1, 10, 30\}$
BC regularization coeff.	$\{0, 0.001, 0.002, 0.005, 0.01, 0.02, 0.05, 0.1, 0.2, 0.3, 0.5, 1.0, 1.5, 3.0, 5.0\}$
Target KL	$\{0.1, 0.5\}$
Optimizer	Adam (wd= 0), AdamW (wd= 1×10^{-6})
<i>Fixed</i>	
Clip range ϵ	0.01
Discount γ	0.99
GAE λ	0.95
Max gradient norm	1.0

and 16 joints for the LEAP hand, together with a point cloud observation of the scene. In the real world, depth observations are collected using an Orbbec Femto Bolt depth camera and projected into the robot base frame to obtain a point cloud, which is cropped to the robot workspace. To account for non-Markovian effects such as robot velocity, we provide each policy with a history of the three previous proprioception observations, the three previous actions, and

the current point cloud. We do not include previous point clouds in the history.

D.2 Action Space

The policy predicts action chunks of length 16, where each action is an absolute joint target for all 23 robot joints. These targets are passed to a joint PD controller for execution. We find that absolute joint targets transfer better to the real world because closed-loop feedback from the controller can compensate for small deviations in physical parameters, such as armature, damping, and joint friction.

D.3 Flow Policy Architecture and Training

We parameterize the velocity field of the flow matching model using a 1D U-Net over action sequences. The input to the U-Net is a noisy action trajectory of horizon $H = 16$, where each action has dimension 23. We condition the U-Net on three sources of information: a point cloud encoding of the scene, a history of robot joint positions, and a history of previous actions. We encode the scene using a lightweight PointNet encoder applied to a 512-point XYZ point cloud. We then concatenate the resulting global point cloud feature with the flattened proprioceptive history and action history, and inject this conditioning vector into the U-Net through FiLM conditioning.

We train the model on real-world human demonstrations using the standard CFM objective. We normalize all joint

TABLE VII
BASE FLOW POLICY HYPERPARAMETERS

Parameter	Value
Policy class	Conditional flow matching
Action representation	Absolute joint position targets
Action dimension	23
Prediction horizon	16
Observation history	4 robot states
Action history	3 previous actions
Point cloud size	512 XYZ points
Point cloud encoder	PointNet
Sequence model	1D U-Net
U-Net channels	[256, 512, 1024]
Conditioning	FiLM scale and shift
Inference steps	5
Batch size	256
Optimizer	Adam
Learning rate	1×10^{-4}
Weight decay	1×10^{-6}
Learning rate schedule	Cosine decay, 500 warmup steps
Training epochs	1000
Validation split	0.05

position inputs and action targets using per-dimension statistics computed from the training set, while keeping point cloud coordinates in metric units in the robot base frame. At inference time, we solve the learned flow using 5 integration steps and execute the resulting absolute joint targets through a joint PD controller. During simulation policy improvement, and at deployment, we run closed loop inference every step, executing only the first action from the chunk. Table VII lists the architecture and training hyperparameters.

D.4 Environment Generation

We construct simulation environments in IsaacLab to match the corresponding real-world task setups. We first combine the Franka arm and LEAP hand URDFs into a single Franka-LEAP robot model. We obtain object assets by scanning real-world objects using ARCode, an off-the-shelf object scanning application that runs on depth-enabled devices such as recent iPhone and iPad Pro models. For tasks involving defective or modified objects, we manually edit the scanned meshes in Blender. For articulated objects, we use Isaac Sim to define the corresponding joints and articulation properties.

After importing the robot and object assets, we configure the simulated scene to match the real-world environment as closely as possible, including object placement, task-relevant geometry, and articulation limits.

D.5 Domain Randomization

A major advantage of simulation is that we can vary physical properties, object initializations, and reset conditions at scale. To obtain policies that are even more robust in the real world, we randomize object scale, mass, friction, and spawn locations during simulation training. We also introduce additional perturbations, such as resetting objects

mid episode and applying external force disturbances, to encourage recovery and retry behaviors.

Because the simulation distribution is intentionally broader and more challenging than the real-world demonstration distribution, the base policies often achieve low initial success rates in simulation despite performing substantially better in the real world. We use this gap as an opportunity for policy improvement: SCORE exposes the base policy to difficult randomized settings and uses sparse rewards to adapt the policy toward more robust behaviors before real-world deployment.

D.6 Asymmetric Residual Flow Steering Implementation

Another advantage of simulation is access to ground-truth state information during training. However, directly training a fully privileged policy would require an additional distillation stage before real-world deployment. To leverage privileged information while preserving deployment compatibility, SCORE uses RFS [12] with an asymmetric actor-critic architecture.

The actor remains non-privileged and only receives observations available at deployment. In particular, we reuse the frozen point cloud encoder from the base CFM policy: the current point cloud is encoded by the frozen PointNet into a 64-dimensional feature vector, which is concatenated with a 4-step history of robot joint positions and a 3-step history of previous actions.

RFS augments the base flow policy by learning steering actions in the flow sampling process. In our implementation, the PPO actor is a single Gaussian policy over a concatenated action vector

$$\xi = [a_r, z],$$

where a_r denotes the residual action component, and z denotes the noise/steering component. Although we refer to these two components separately, they are produced by a single MLP actor as one concatenated vector with a single diagonal Gaussian distribution. We therefore write the actor as

$$\xi \sim \pi_{\text{steer}}(\xi \mid o),$$

rather than as two separate policies.

The critic is privileged and is used only during simulation training. It receives simulator state features that are unavailable or unreliable in the real world such as object pose, target pose, and other task-specific state terms. We exclude raw visual observations such as point clouds or RGB images from the critic input. This asymmetric design allows the value function to use ground-truth simulator information for lower-variance advantage estimation, while keeping the actor restricted to deployment-compatible observations.

We estimate advantages using GAE with the privileged critic $V_\psi(s)$, and update the actor using the standard clipped PPO objective:

$$J_{\text{actor}}(\theta) = \mathbb{E}_{(o,s,\xi) \sim \mathcal{B}} \left[\min(\rho_\theta \hat{A}, \text{clip}(\rho_\theta, 1 - \varepsilon, 1 + \varepsilon) \hat{A}) \right],$$

where

$$\rho_\theta = \frac{\pi_{\text{steer}}(\xi | o)}{\pi_{\text{steer}}^{\text{old}}(\xi | o)}$$

is the standard PPO importance ratio over the joint concatenated action $\xi = [a_r, z]$, and $\hat{A}$ is the GAE advantage estimate computed using the privileged critic. Since only the critic depends on privileged simulator state, the learned actor can be deployed directly in the real world without distillation.

D.7 Deployment of SCORE in the Real World

At deployment time, we directly use the actor learned during simulation together with the original frozen base policy. The learned actor predicts a concatenated steering action consisting of a noise component and a residual component at test time conditioned on the same observations as the base policy.

We use the predicted noise component as the input noise for the base CFM sampler. After integrating the learned velocity field to denoise the action trajectory, we add the predicted residual component to the resulting action. This produces a modified action trajectory that remains anchored to the base policy while allowing the RFS actor to steer the generated behavior toward higher-reward actions.

Although the base policy predicts an action chunk of horizon $H = 16$, we execute only the first action in the predicted chunk on the real robot. We then replan at the next control step using the latest observation and history.

E. QUALITATIVE ANALYSIS

E.1 Single-Task Policy Improvement and Failure Modes

We provide qualitative examples of common failure modes for the baselines and our method. These examples illustrate that high success in simulation does not necessarily imply safe or reliable real-world deployment. We observe two broad failure patterns. Weakly constrained methods can improve aggressively in simulation by exploiting simulator-specific dynamics or drifting away from the real-world behavior distribution. More conservative methods can preserve transferability, but may also preserve slow, imprecise, or suboptimal behaviors from the base policy. In contrast, SCORE targets a middle ground: it improves the policy in simulation while constraining the learned behavior to remain within the action support of the real-world base policy.

FPO. FPO often improves simulated success by moving outside the support of the real-world base policy. This can produce behaviors that appear effective in simulation but are brittle or unsafe on hardware. In the Lightbulb Screw task shown in Fig. 17, the policy learns an abnormal twisting strategy that applies excessive force to the lightbulb fixture, causing the stand to be pulled off the table rather than reliably rotating the bulb. In Soccer Push, FPO learns to strike the table and trap the ball using simulator-specific contact dynamics. While this behavior can achieve the simulated objective, it is unsafe to deploy on real hardware and does not reflect the intended task strategy. In Credit Card Pick, the policy learns to swipe the card off the shelf, exploiting differences between simulated and real contact dynamics.

Similarly, in Bottle Grasp, we observe a smaller but still noticeable drift from the base behavior: the middle finger twists inward, producing grasps that are less stable in the real world. Across these examples, FPO’s failures are consistent with a lack of support preservation: the optimized policy discovers high-reward behaviors in simulation that are not reliably transferable.

RialTo. RialTo is more conservative than FPO because it explicitly regularizes policy improvement toward the base behavior distribution. This makes it less likely than FPO to discover extreme simulator-specific reward-hacking strategies, but it also limits how much the policy can improve. As shown in Fig. 18, RialTo can still produce small but harmful deviations from real-world behavior. In Soccer Push, the fingers drag on the table and damage the pinky finger. In Bottle Grasp, RialTo also learns to invert the middle finger in an unnatural way, producing unstable grasps on hardware. In Dishrack Place, RialTo learns reward-hacking behavior in simulation by moving the dishrack center close enough to the success region defined in Table III, without actually placing the plate inside the dishrack. These examples reflect the central tradeoff of distributional regularization: weak regularization permits unsafe drift, while strong regularization preserves suboptimal behaviors from the base policy.

Residual RL. Residual policy methods improve the base policy by learning bounded corrective actions on top of the original behavior. This can help when the base policy already chooses a reasonable strategy and only requires local refinement. However, in our experiments, failures often occur when the base policy selects a poor or suboptimal behavior mode. In these cases, the bounded residual correction cannot steer the policy toward a substantially different strategy. This limitation is most visible in tasks where the base policy has low performance, such as Ball Pour and Dishrack Place. Residual RL can improve local precision, but it remains constrained by the behavior selected by the base policy.

SCORE. SCORE can fail when the base policy’s support does not contain a successful behavior for a task, as discussed in the limitations section (Sec. VI). In our experiments, however, the observed failures are primarily due to task-level precision requirements and contact-rich dynamics rather than large support drift. Fig. 19 shows successful SCORE rollouts and Fig. 20 its failure modes. In Credit Card Pick, small differences in contact can cause the card to fall from the shelf, and successful grasping requires precise coordination between the fingers to slide under and lift the card. In Ball Pour, slight misalignment of the cup or pouring too quickly can cause the balls to spill outside the target container. In Soccer Push, the nontrivial dynamics of the squishy real-world ball make accurate pushing difficult, and overly hard or poorly aligned pushes can result in missed goals. In Dishrack Place, failures are rare, but can occur when the robot places the plate in an incorrect slot or contacts the dishrack geometry during insertion. Overall, SCORE failures are mostly caused by contact sensitivity and task precision, rather than simulator-specific reward hacking or unsafe behaviors outside the real-world policy support.

E.2 Multi-Task Steering and Cross-Task Behavior Reuse

We also provide qualitative examples for the multi-task steering experiment. In this setting, a single base policy is trained on demonstrations from Cube Pinch, Bottle Grasp, and Credit Card Pick. This broader dataset expands the behavior support of the base policy, but also introduces interference between task-specific contact strategies. As shown in Fig. 21, the base policy sometimes applies behaviors from one task in another setting, such as using Credit Card Pick-like high grasps for object grasping or using grasping motions that are poorly matched to the current object and reset distribution. This causes the multi-task base policy to underperform the corresponding single-task policies, despite being trained on a broader dataset.

After simulation steering, SCORE can more reliably select task-appropriate behaviors from the shared prior. More importantly, the broader multi-task prior enables useful cross-task behavior reuse, as shown in Fig. 22. For example, when Cube Pinch is evaluated under the wider Bottle Grasp reset distribution, single-task SCORE often lacks sufficient support for the broader object locations, while multi-task SCORE can reuse the wider grasping coverage learned from Bottle Grasp. Similarly, contact-rich sliding behavior from Credit Card Pick can help recover cube grasps outside the original Cube Pinch randomization. These examples suggest that multi-task data can expand the support available to SCORE, but that steering is necessary to exploit this broader prior reliably.



Fig. 17. **FPO failure modes.** FPO can exploit simulator-specific dynamics and drift outside the real-world policy support, producing unsafe or non-transferable behaviors.

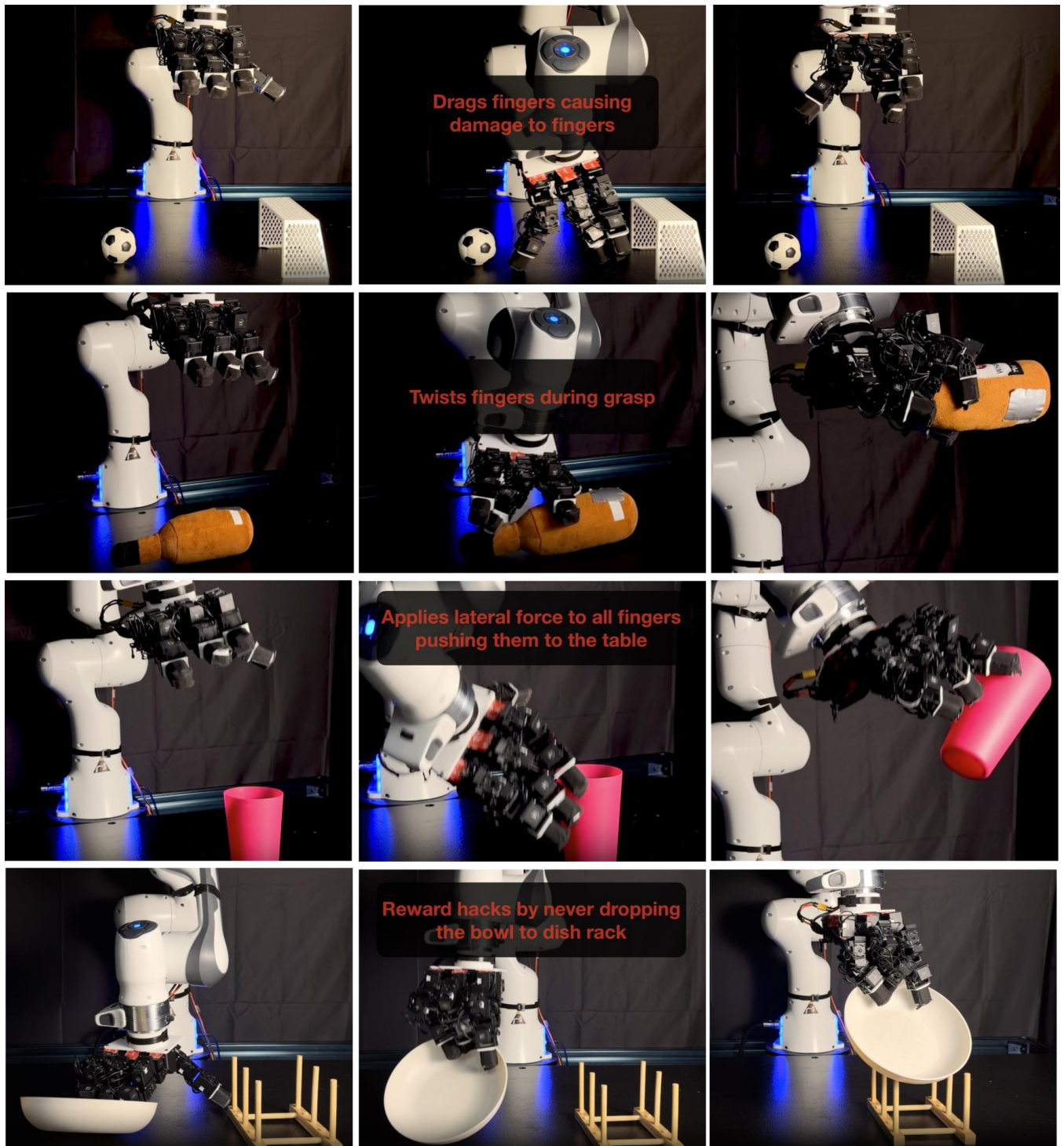


Fig. 18. **RialTo failure modes.** BC regularization can limit the amount of drift from the support of the real-world policy, but can also limit improvement and retain imprecise base-policy behaviors.

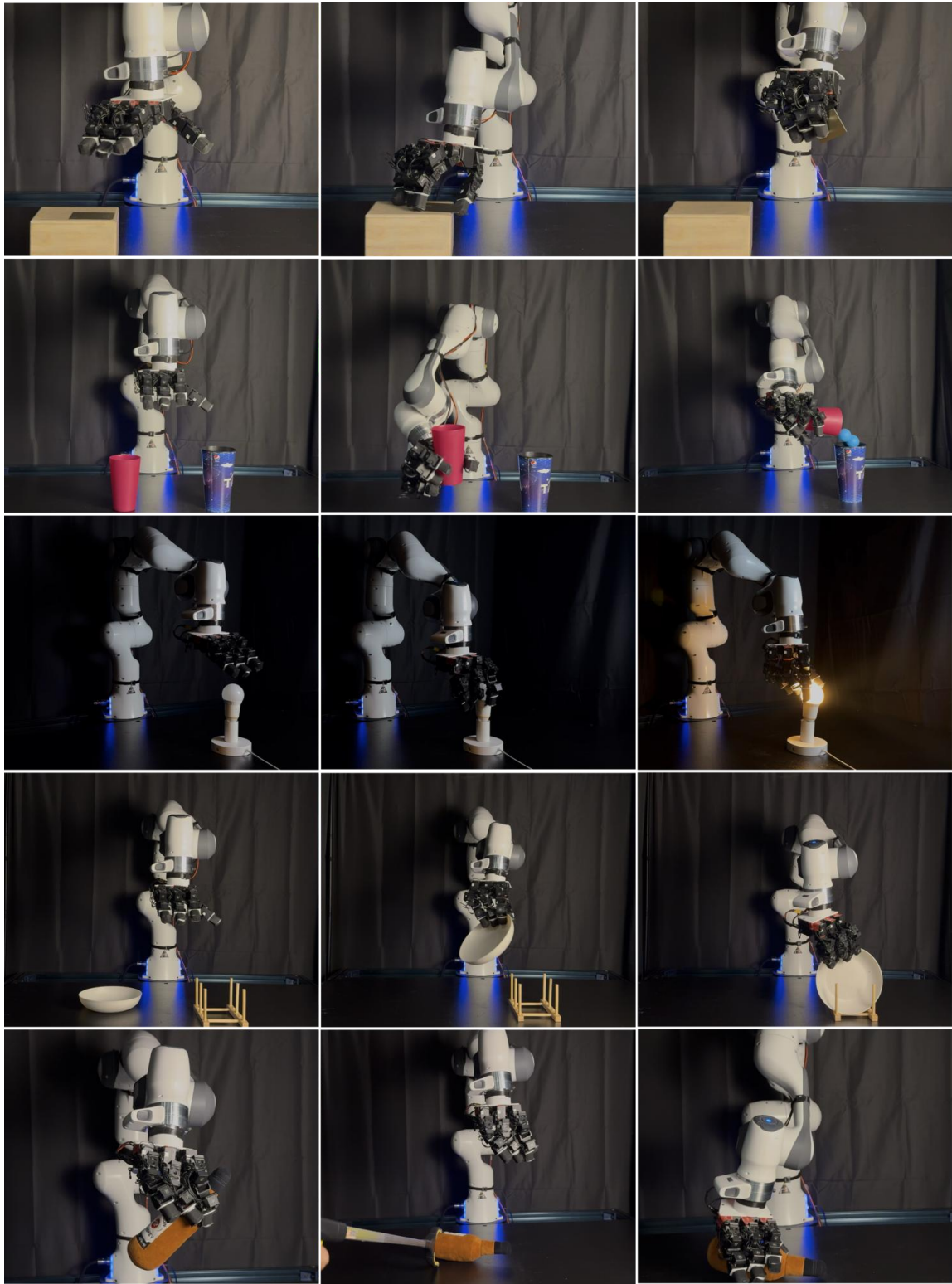


Fig. 19. **SCORE successful rollouts.** SCORE improves task performance while maintaining real-world-feasible behaviors within the base policy support.



Fig. 20. **SCORE failure modes.** SCORE failures are primarily caused by contact sensitivity and task precision demands, rather than unsafe support drift.

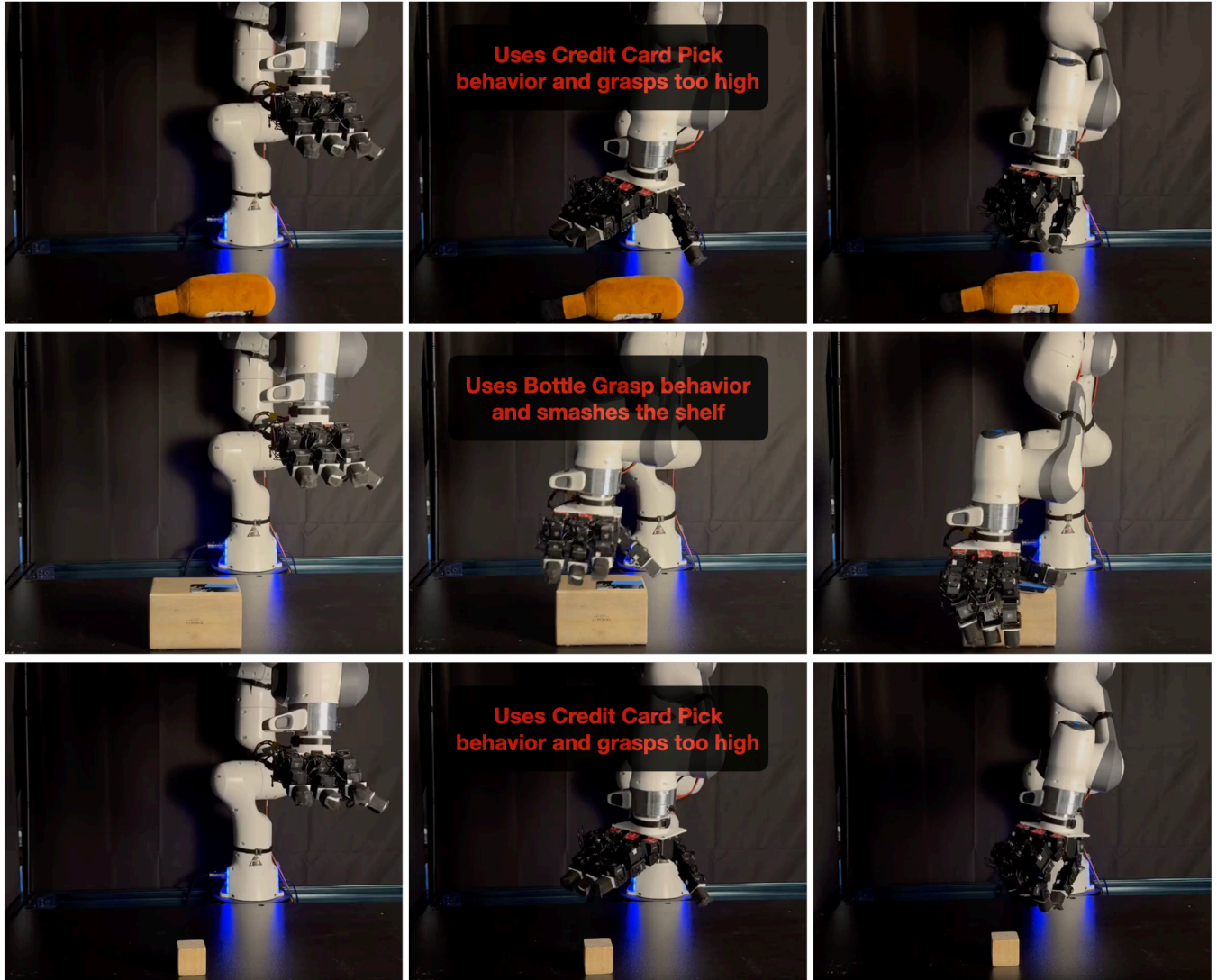


Fig. 21. **Multi-task base policy failure modes.** Although multi-task training expands the behavior support of the base policy, direct deployment can suffer from interference between task-specific behaviors. The multi-task base policy sometimes applies behavior modes from the wrong task, such as Credit Card Pick-like high grasps during object grasping or Bottle Grasp-like motions during Credit Card Pick, leading to unstable grasps or collisions with the shelf.

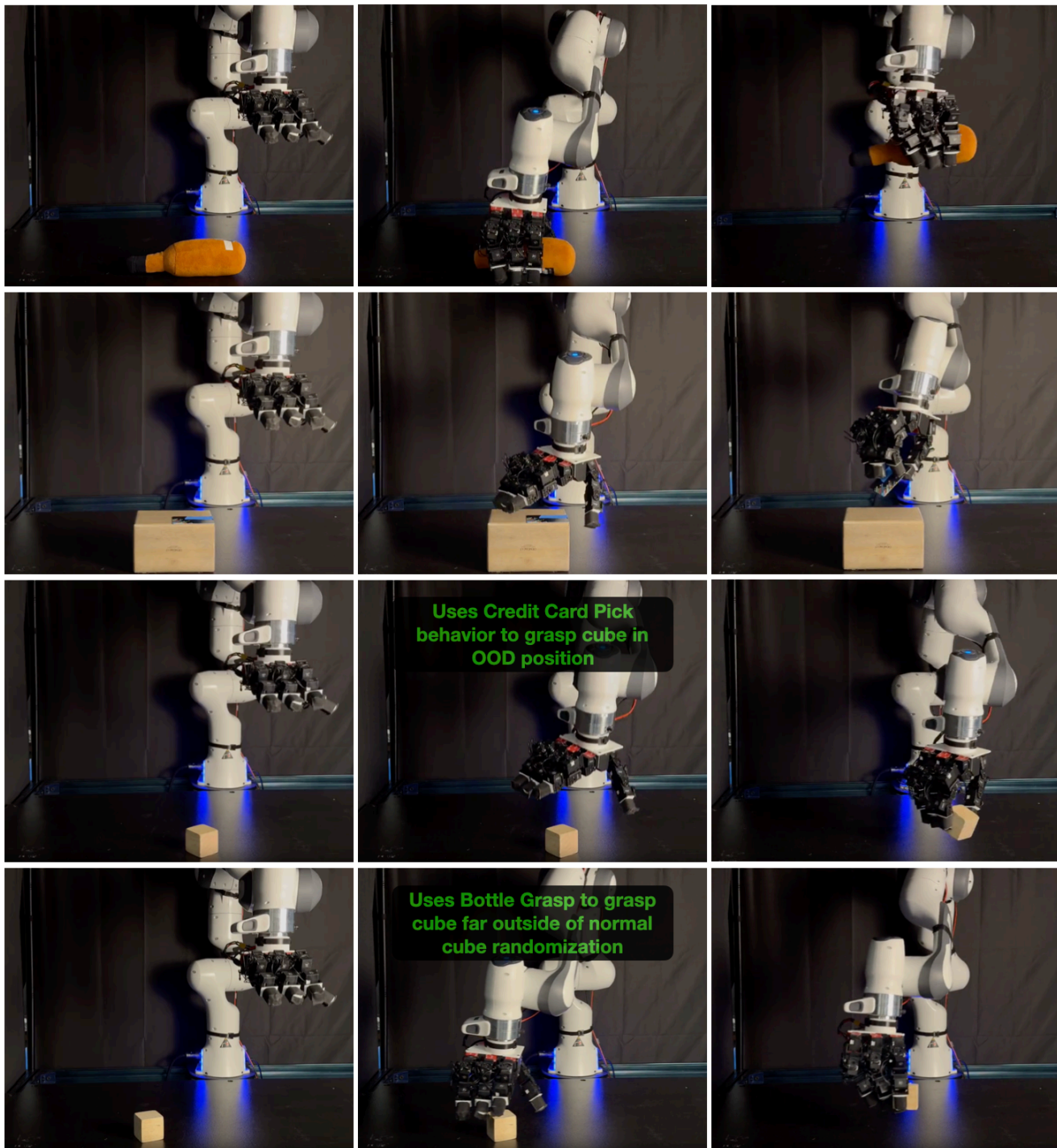


Fig. 22. **Multi-task SCORE enables cross-task behavior reuse.** After steering a shared multi-task prior, SCORE can select task-appropriate behaviors while also reusing useful strategies across tasks. The top two rows show successful Bottle Grasp and Credit Card Pick executions. The bottom rows show Cube Pinch under broader object placements, where multi-task SCORE can reuse Credit Card Pick-like sliding behavior and Bottle Grasp-like wide-reset grasping behavior to recover cubes outside the original Cube Pinch randomization.